Universidade Estadual da Paraíba

Centro de Ciências e Tecnologia - CCT

Departamento de Computação

Curso de Ciência da Computação

Disciplina: Compiladores

INTRODUÇÃO

Professora: Kézia de Vasconcelos Oliveira Dantas

kezia.vasconcelos@servidor.uepb.edu.br

Agenda

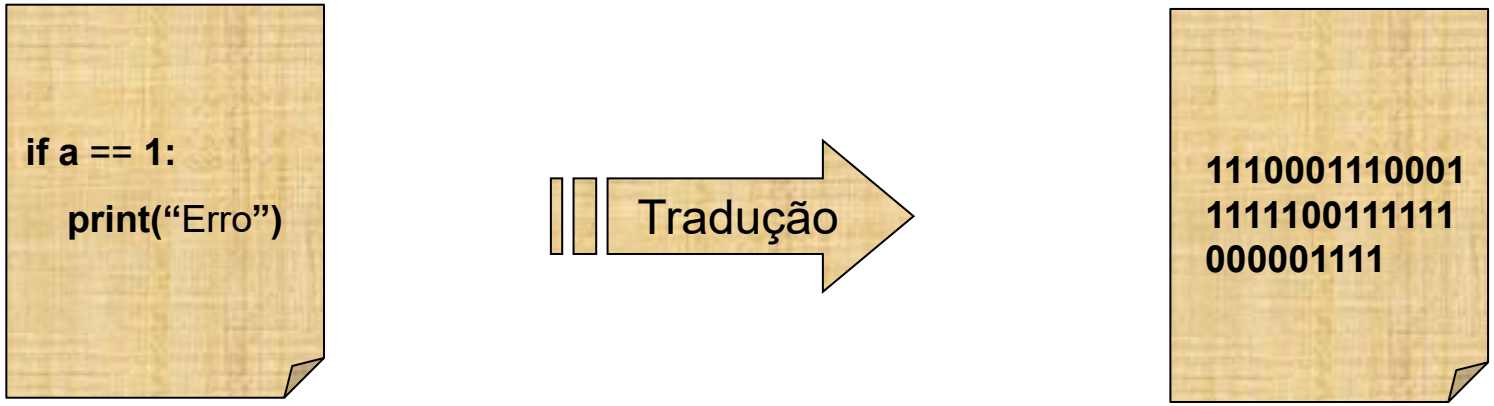
- Introdução
- Processadores de Linguagem
- Histórico
- Aplicações

Linguagens de Programação

- Linguagens de programação X resolução de problemas

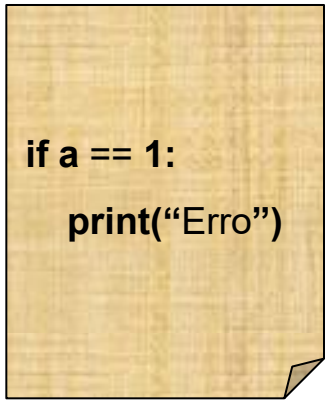
Linguagens de Programação

- Para que se tornem operacionais, os programas escritos em linguagens de programação devem ser traduzidos para um formato que lhe permita ser executado por um computador.

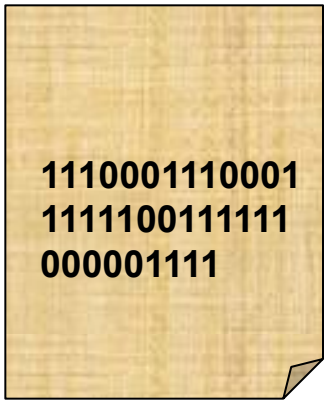


Processadores de Linguagem

Um compilador é um programa que recebe como entrada um programa em uma linguagem e o traduz para um programa equivalente em outra linguagem. Aho. Et al., 2007.



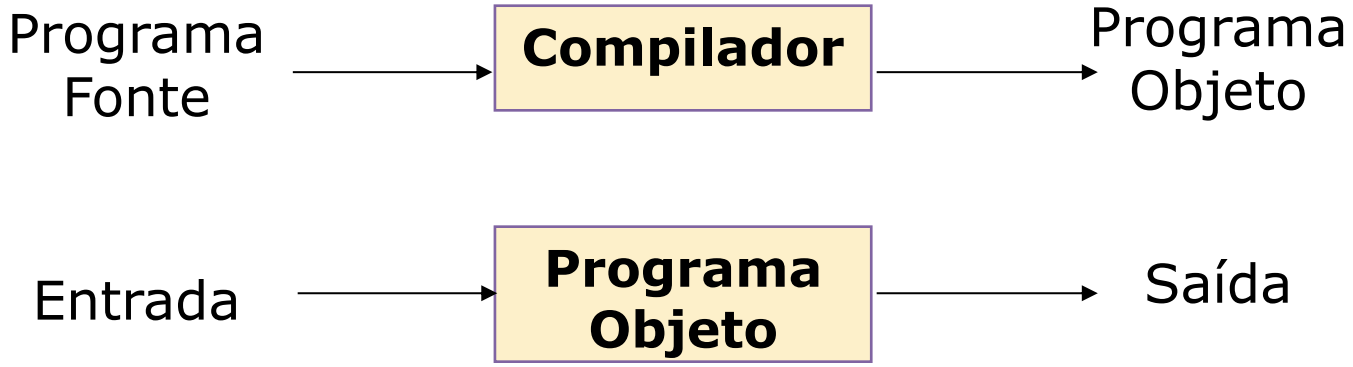
Linguagem Fonte



Linguagem Objeto

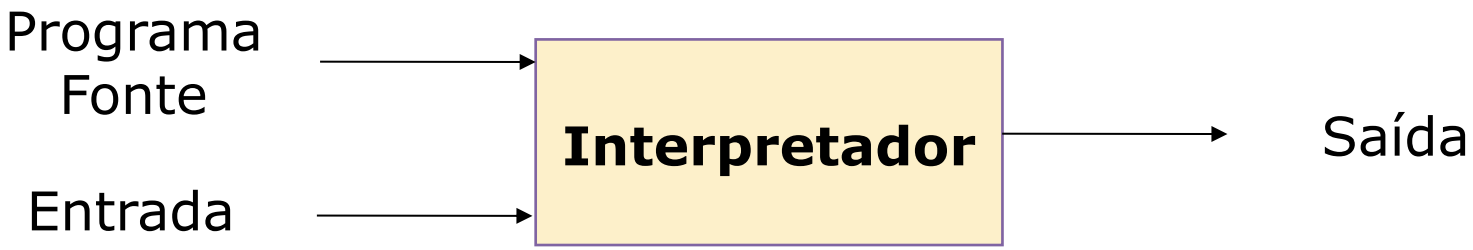
Processadores de Linguagem

- A execução de um programa escrito em linguagem de programação de alto nível é basicamente um processo de dois passos.



Processadores de Linguagem

- Interpretador
  - É outro tipo de processador de linguagem.



Processadores de Linguagem

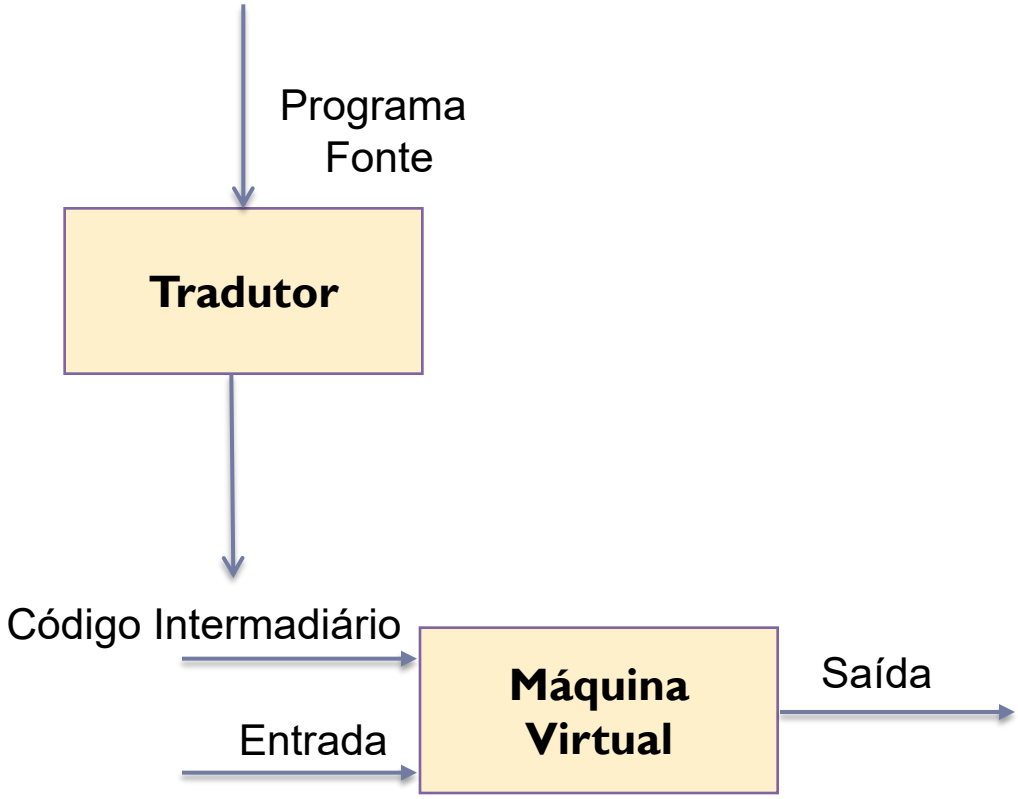
- Compilador:
  - Traduz o programa para linguagem de máquina
  - É mais rápido no mapeamento de entradas para saídas
  - Ex.: C, Pascal, Cobol, etc

Processadores de Linguagem

- Interpretador:
  - É mais lento no mapeamento de entradas para saídas
  - A interpretação facilita a depuração, pois a execução do programa fonte é feito instrução por instrução
  - Ex.: PHP, Scheme, etc

Processadores de Linguagem

- Híbrido:
  - Combina compilação e interpretação.
  - Ex.: Perl, Java, .NET, etc.



Processadores de Linguagem

- JITs – Compiladores Just-In-Time
  - É uma técnica de interpretação usada para aumentar o desempenho na execução de programas.
  - Esta técnica permite que ambientes de execução compilem código em tempo de execução para código de máquina real

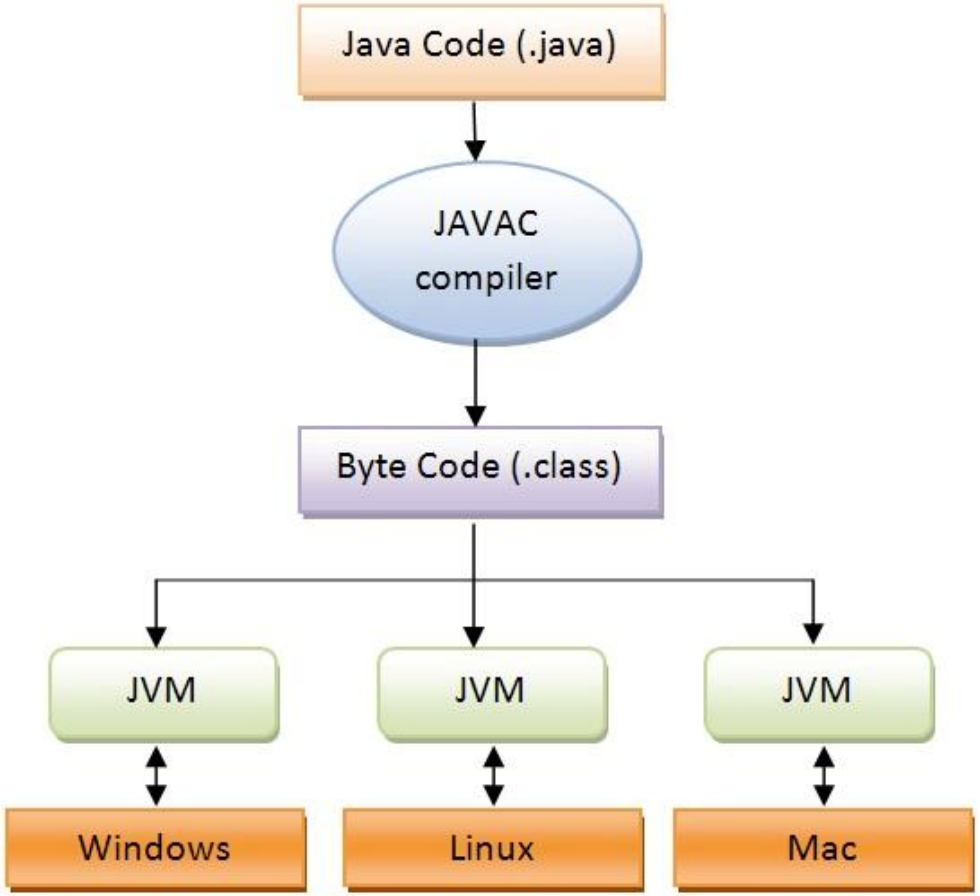
Processadores de Linguagem

- JITs – Compiladores Just-In-Time
  - Ex.: Java e .NET
  - Compiladores similares aos da abordagem híbrida, mas ao invés de interpretarem o bytecode, traduzem (compilam) este para código de máquina nativo enquanto o executa
    - Tarefas custosas, como: verificação e otimização são realizadas na compilação para bytecodes
    - Durante a execução do programa, o compilador JIT realiza apenas a tradução de bytecode para código de máquina
      - Mais simples e rápido

Processadores de Linguagem

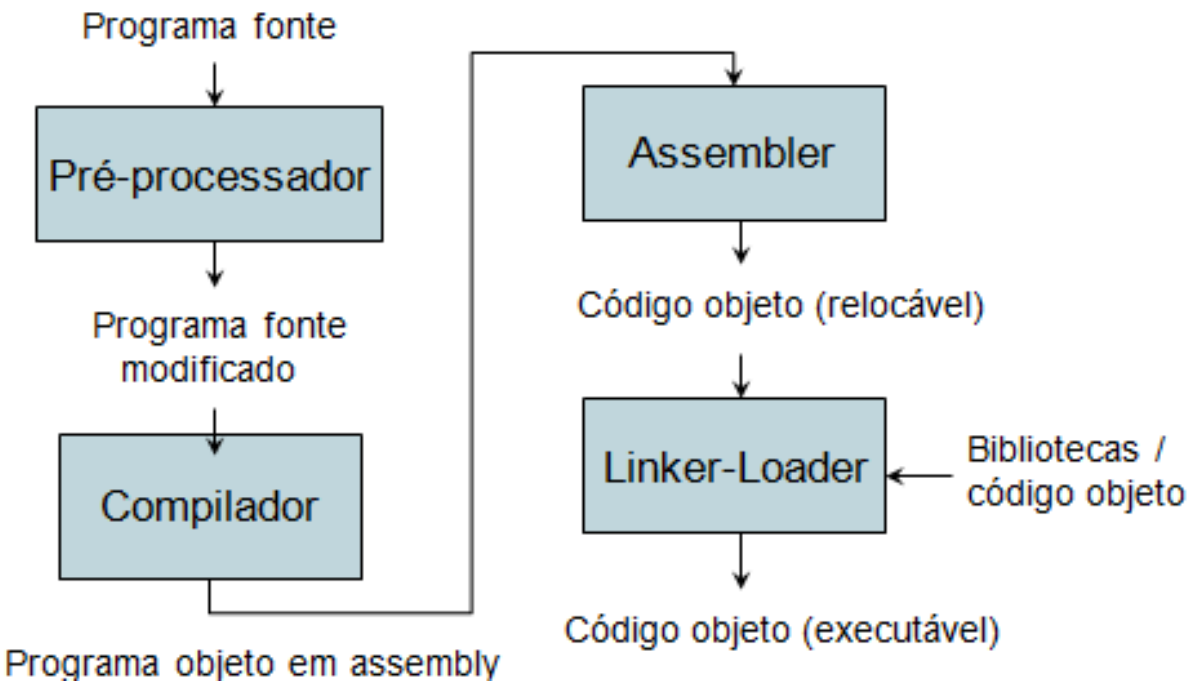
- JITs – Compiladores *Just-In-Time*
  - Para este tipo de compilador cada trecho de código é traduzido no instante em que está para ser executado pela primeira vez, daí derivando o nome "just-in-time"

Máquina Virtual de Java



Fonte: http://www.devmedia.com.br/entenda-como-funciona-a-java-virtual-machine-jvm/27624

Processo de Compilação



Processo de Compilação

- Pré-processador: coleta os módulos, em arquivos separados, que compõem o programa fonte; Expande macros para as construções da linguagem fonte.

- Compilador: a partir do resultado do pré-processador, produz um programa em linguagem assembly.

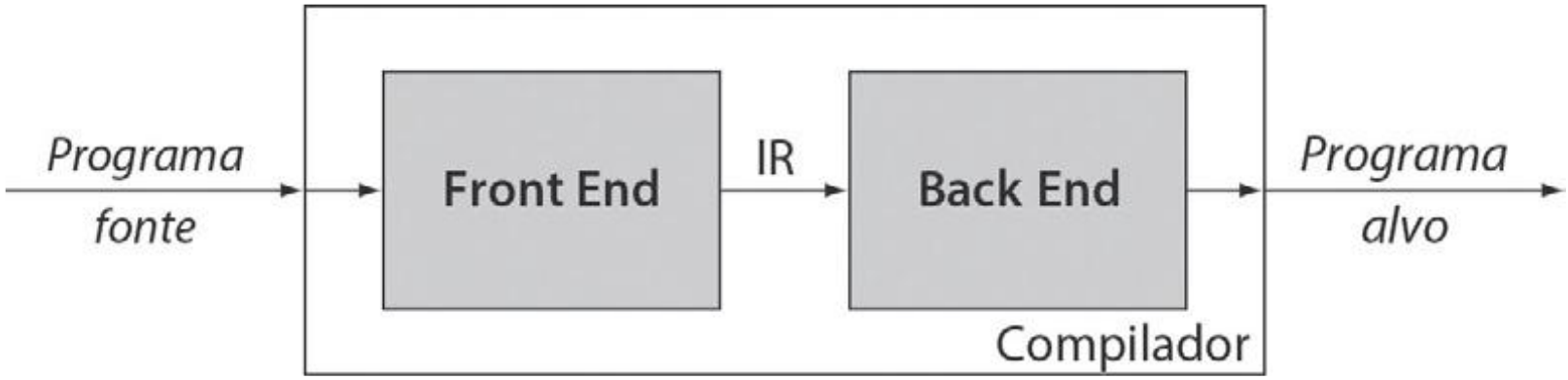
- O assembler (montador) produz código de máquina relocável a partir da saída do compilador
  - Relocação é o processo de atribuir endereços de memória para as várias partes do programa e ajustar o código e dados no programa para refletir os endereços atribuídos

Processo de Compilação

- O linker resolve endereços de memória externos ao programa fonte (referências a arquivos externos e/ou bibliotecas)

- O loader reserva espaço para armazenar o programa (completo) na memória para a execução

Estrutura de um compilador

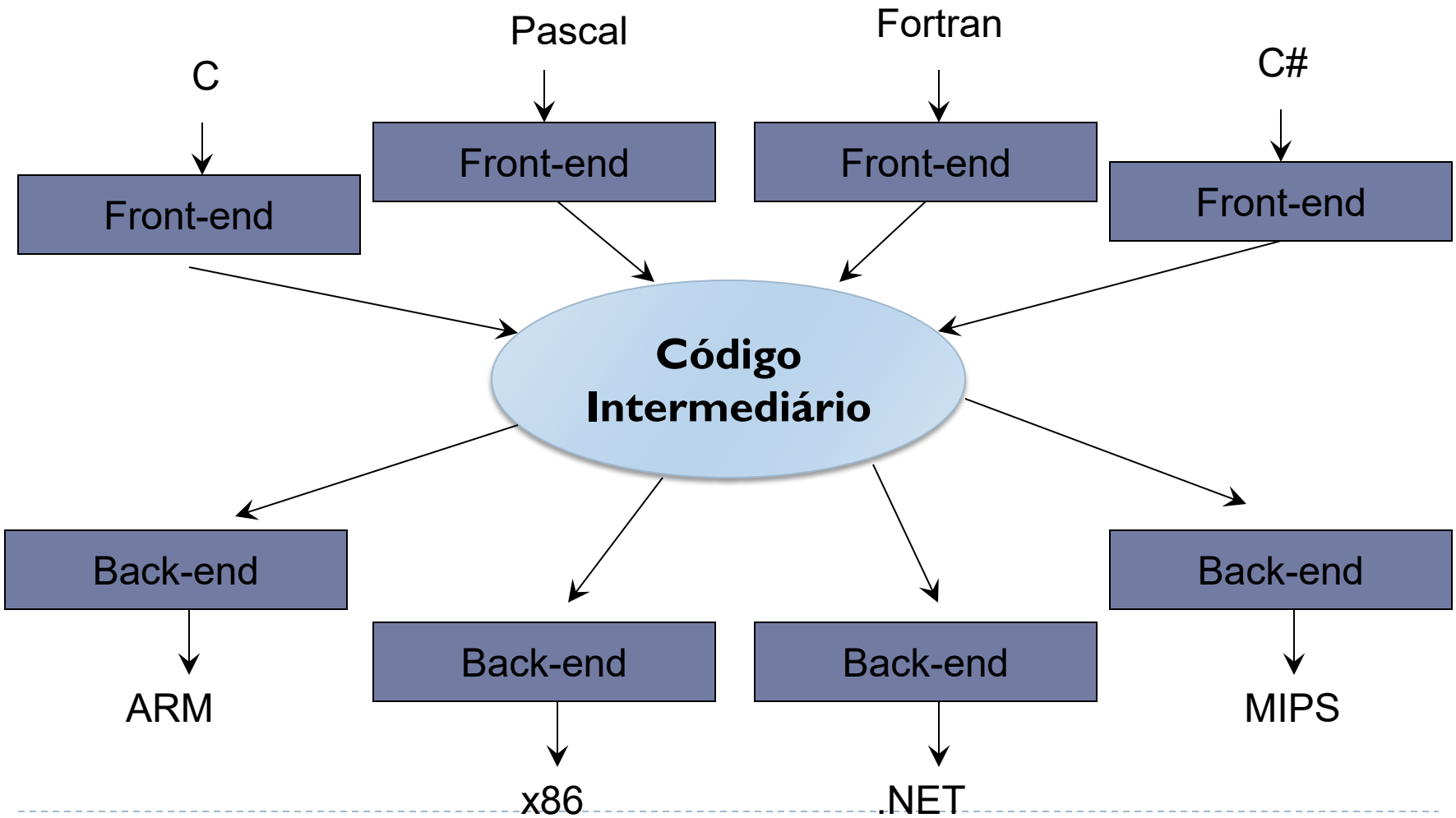


Estrutura de um compilador

- Front-end (análise)
  - Divide programa fonte em partes e impõe uma estrutura gramatical
    - Criar representações intermediárias do programa
  - Verifica presença de certos tipos de erros
    - Sintáticos e semânticos
  - Armazena informações na tabela de símbolos

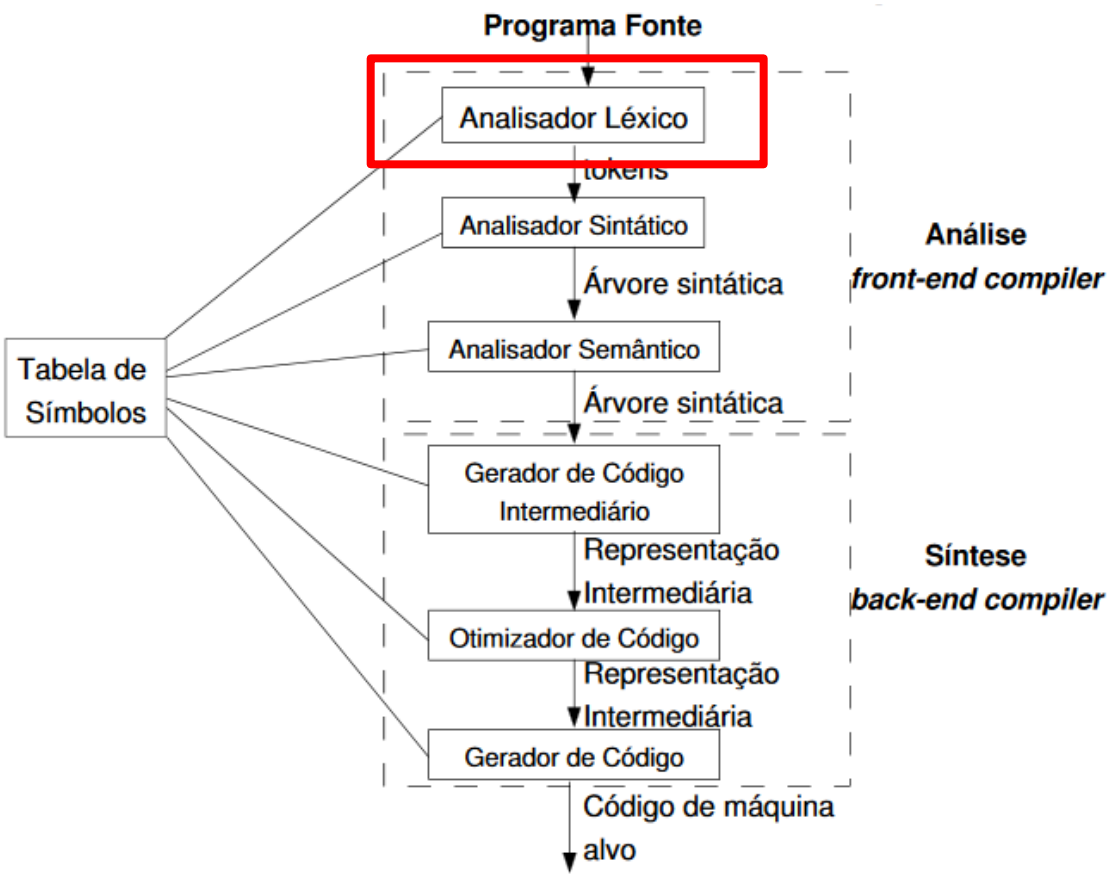
- Back-end (síntese)
  - Constrói o programa objeto a partir de representações intermediárias e da tabela de símbolos

Separação entre front-end e back-end para criação de múltiplos compiladores



Fases de um compilador

- O processo de compilação é desenvolvido em uma sequencia de fases

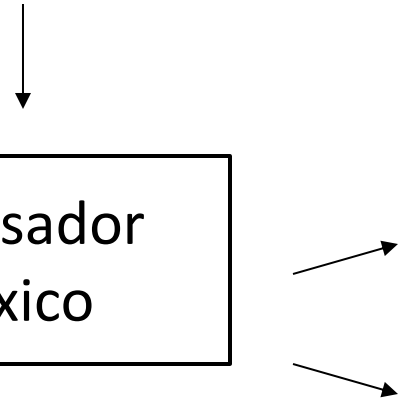


Fases de um compilador

- Analisador léxico (scanner):
  - Lê o programa fonte caracter a caracter, agrupando a sequencia de caracteres lidos em grupos de símbolos (tokens)
  - Os símbolos são as palavras-chaves, nomes (identificadores) de variáveis, procedimentos, operadores e símbolos de pontuação

Fases de um Compilador

```
position = initial + rate * 60
```



Analisador Léxico

<**id**, 1>, <=>, <**id**, 2>, <+>,
<**id**, 3>, <*>, <60>

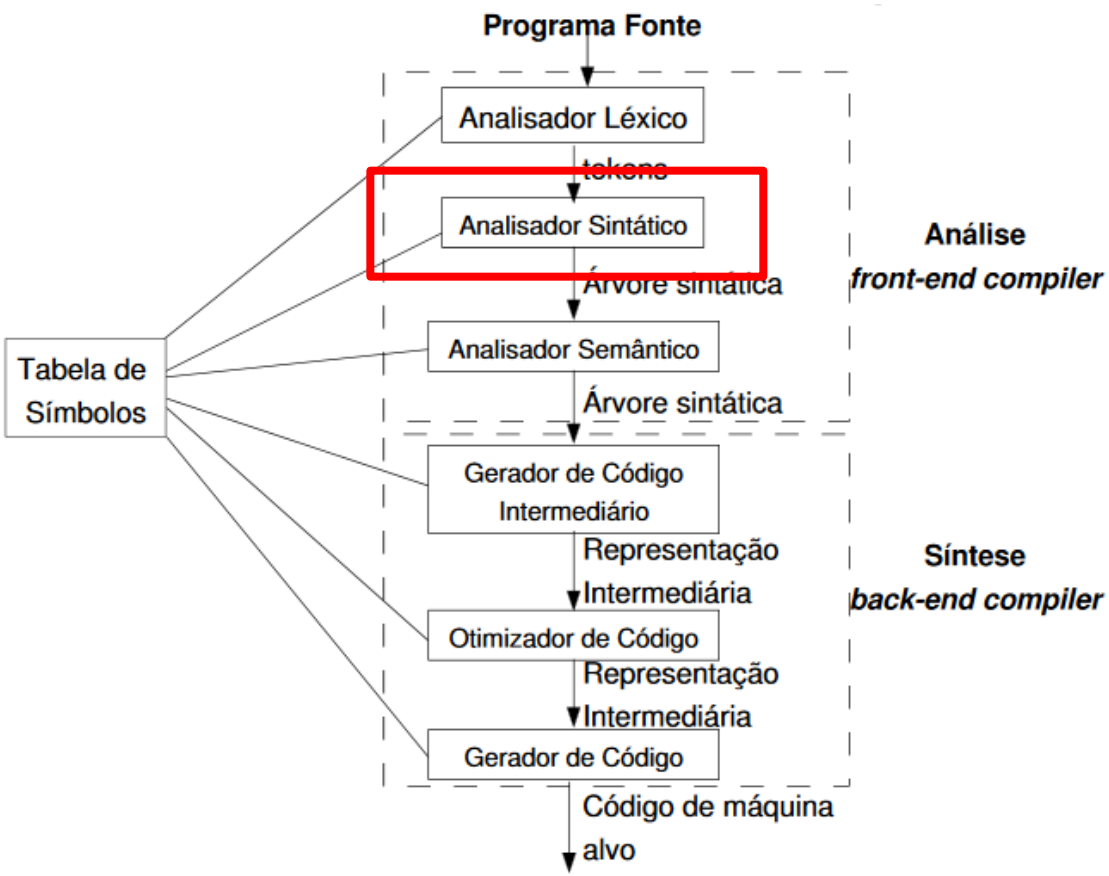
Tabela de Símbolos

	nome	**tipo**	
1	`position`	-	...
2	`initial`	-	
3	`rate`	-	
...			

Fases de um compilador

- Analisador léxico (scanner):
  - O projetista do compilador caracteriza o analisador léxico através de expressões regulares (ERs).
  - A geração do analisador léxico é automática a partir da definição das ERs.
    - Ex.: JFLEX, CSLEX

Fases de um Compilador



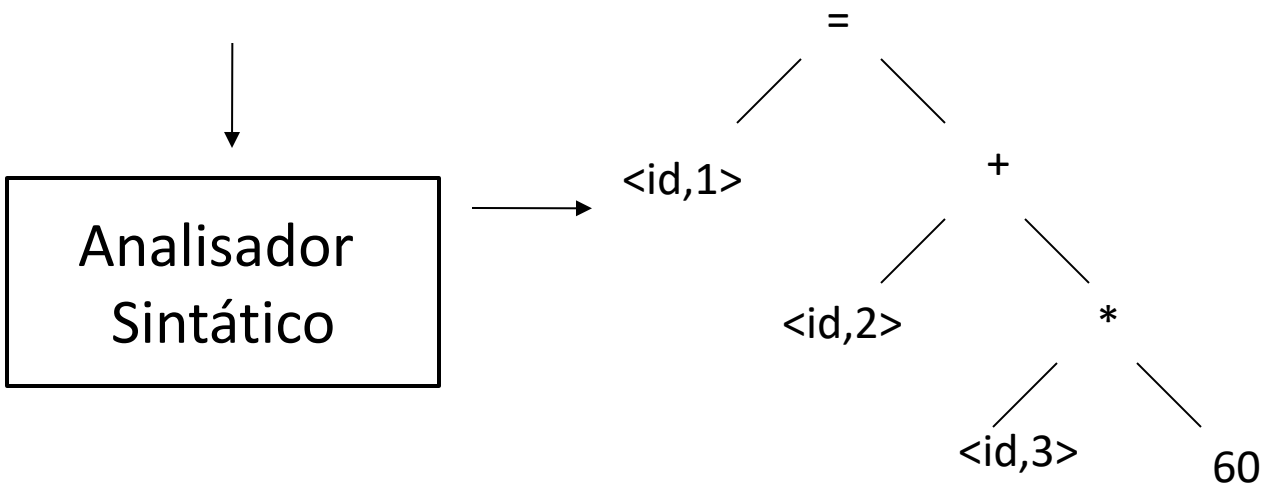
Fases de um Compilador

- Analisador sintático(scanner):
  - A partir dos tokens cria uma estrutura em árvore (árvore sintática) que representa a estrutura gramatical do programa

Fases de um Compilador

- Analisador sintático(scanner): Exemplo

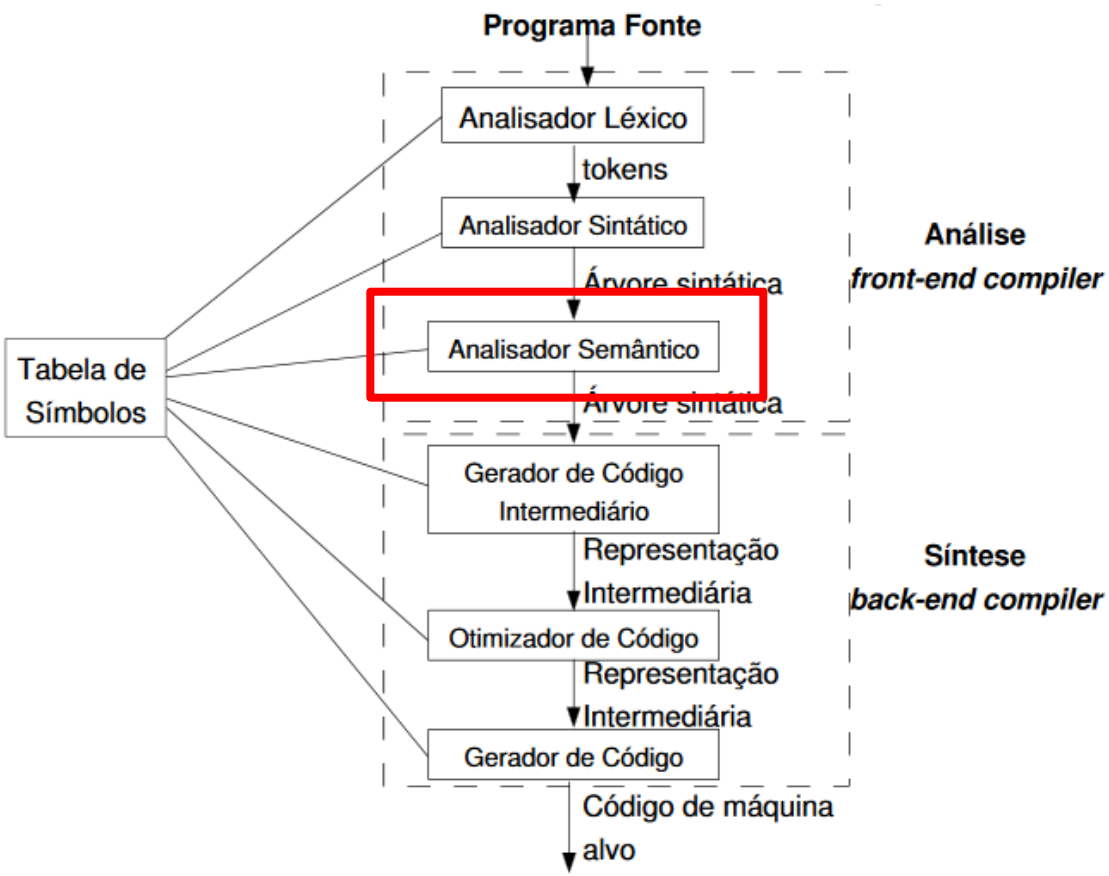
<**id**, 1>, <**=**>, <**id**, 2>, <**+**>,
<**id**, 3>, <*****>, < 60>



Fases de um Compilador

- Analisador sintático(scanner):
  - Gramática livre de contexto (BNF) caracteriza a linguagem.
  - A geração do parser a partir de uma BNF é automática.
    - Ex: JavaCC, JCup, yacc, etc.

Fases de um Compilador



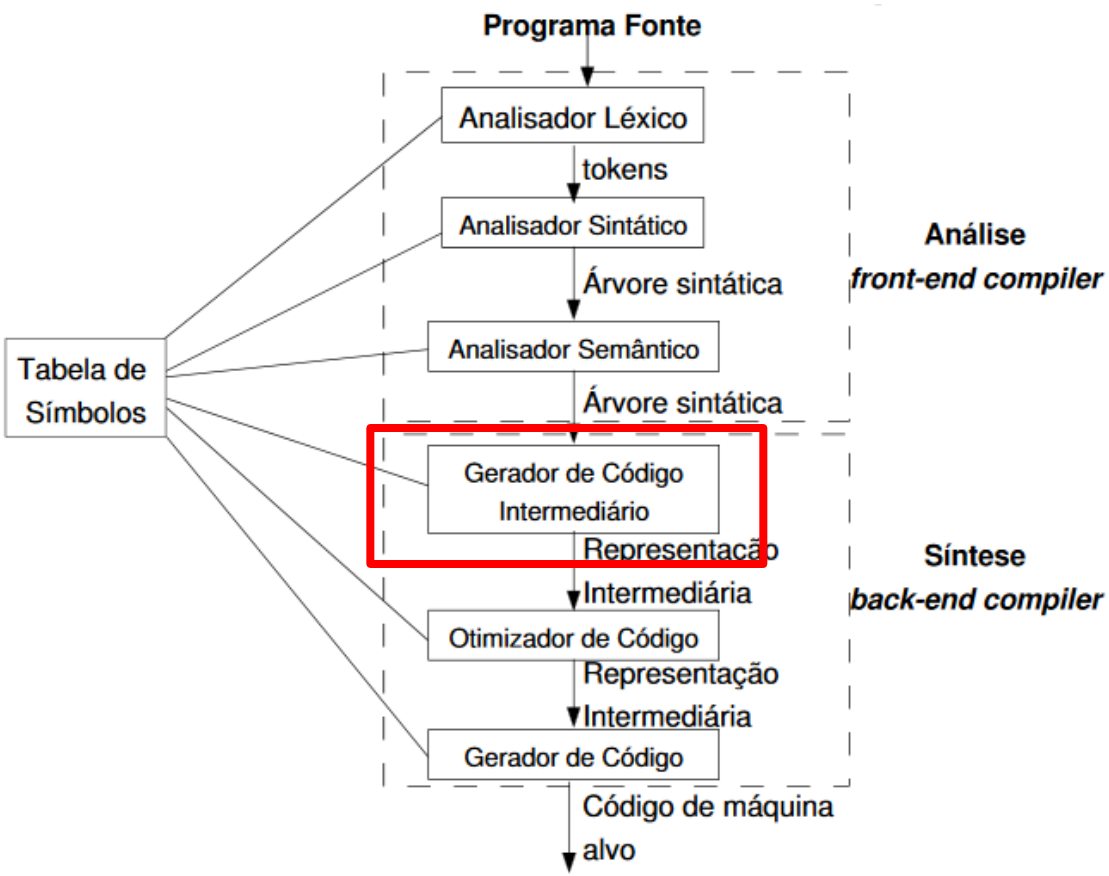
Fases de um Compilador

- Analisador semântico verifica a consistência semântica do programa fonte com a definição da linguagem.

- Faz a verificação de tipos
  - Verifica se cada operador possui operandos compatíveis

- Verifica se a linguagem permite coerções
  - position = initial + rate * 60
    - initial e rate forem float e 60 inteiro -> haverá conversão automática

Fases de um Compilador



Fases de um Compilador

- O gerador de código intermediário:
  - Usa as estruturas produzidas pelo analisador sintático e verificadas pelo analisador semântico para criar uma sequencia de instruções simples dita código intermediário
  - Código intermediário: está entre a linguagem de alto nível e a linguagem de baixo nível

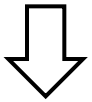
Fases de um Compilador

- O gerador de código intermediário:
  - Exemplo: código de três endereços
    - Regras:
      - Cada instrução de atribuição possui no máximo um operador do lado direito. Ex: t1 = id4 + id2
      - O compilador precisa gerar um nome temporário para guardar o valor computado por uma instrução
      - Uma instrução pode ter menos de três operandos.Ex: t2 = id5
    - Vantagens: fácil de ser produzida e traduzida para uma máquina alvo

Fases de um Compilador

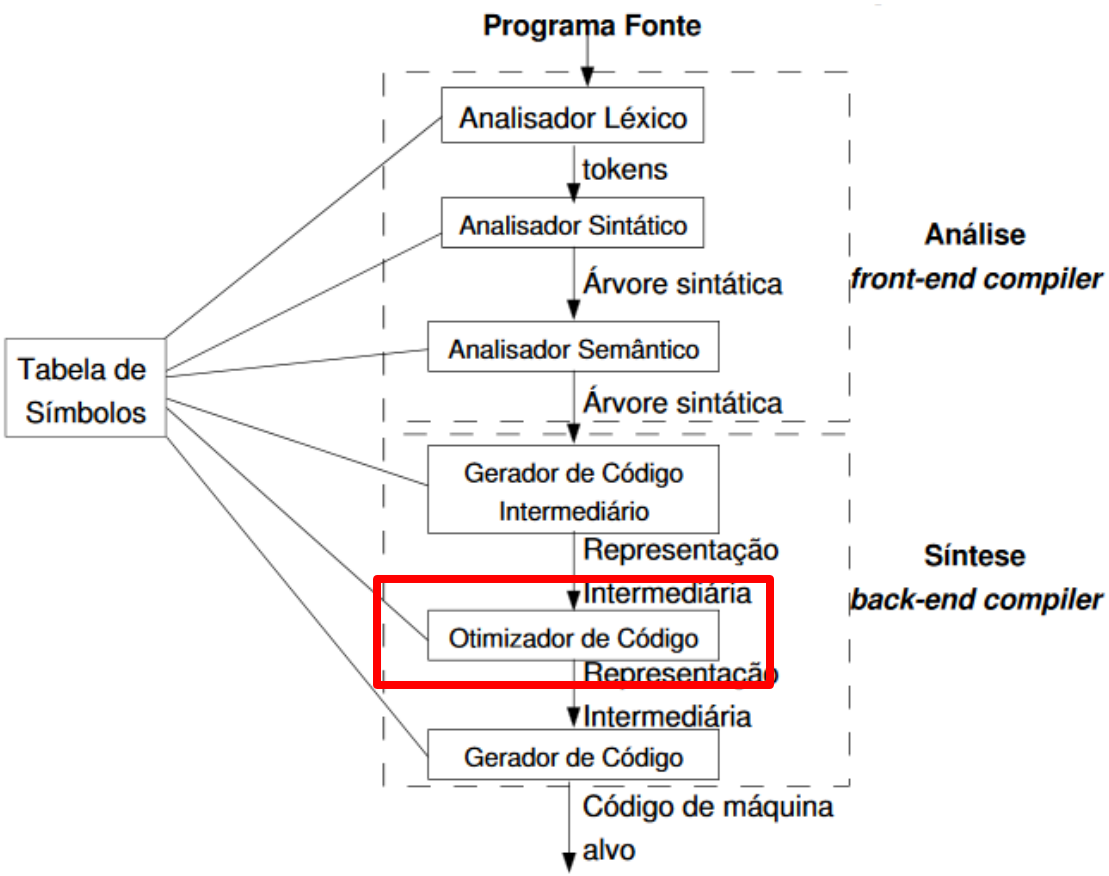
- O gerador de código intermediário: Exemplo - 3 endereços)

```
id1 = id2 + id3 * inttofloat(60)
```



```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Fases de um Compilador

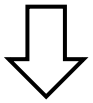


Fases de um Compilador

- Otimizador de código:
  - Independente de máquina
  - É um módulo opcional (presente na grande maioria dos compiladores) que objetiva melhorar o código intermediário de modo que o programa objeto produzido ao fim da compilação seja menor e/ou mais rápido ou consuma menos energia
    - Quanto mais otimizações maior o tempo gasto nesta fase
  - A saída do otimizador de código é um novo código intermediário

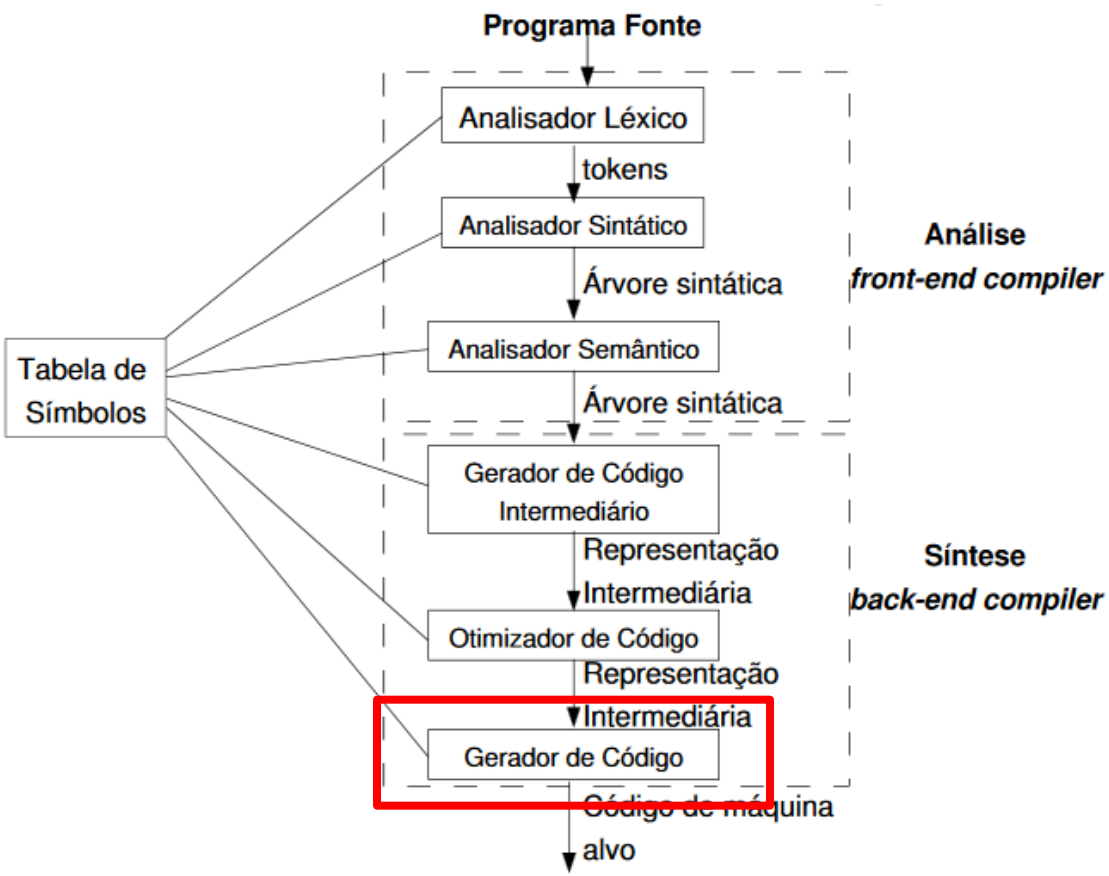
Fases de um Compilador

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```



```
t2 = id3 * 60.0
id1 = id2 + t2
```

Fases de um Compilador



Fases de um Compilador

- O gerador de código:
  - Produz o código objeto final, tomando decisões com relação à alocação de espaço para os dados do programa, selecionando a forma de acessá-los, definindo que registradores da CPU serão usados, etc
  - Projetar um gerador de código que produza programas objeto verdadeiramente eficientes é uma das tarefas mais difíceis no projeto de um compilador
  - A maioria dos compiladores atuais tem, como parte integrante um módulo adicional de otimização de código dependente de máquina que tem por objetivo melhorar o código de máquina produzido para melhor aproveitar os recursos específicos da arquitetura para a qual foi gerado

Fases de um Compilador

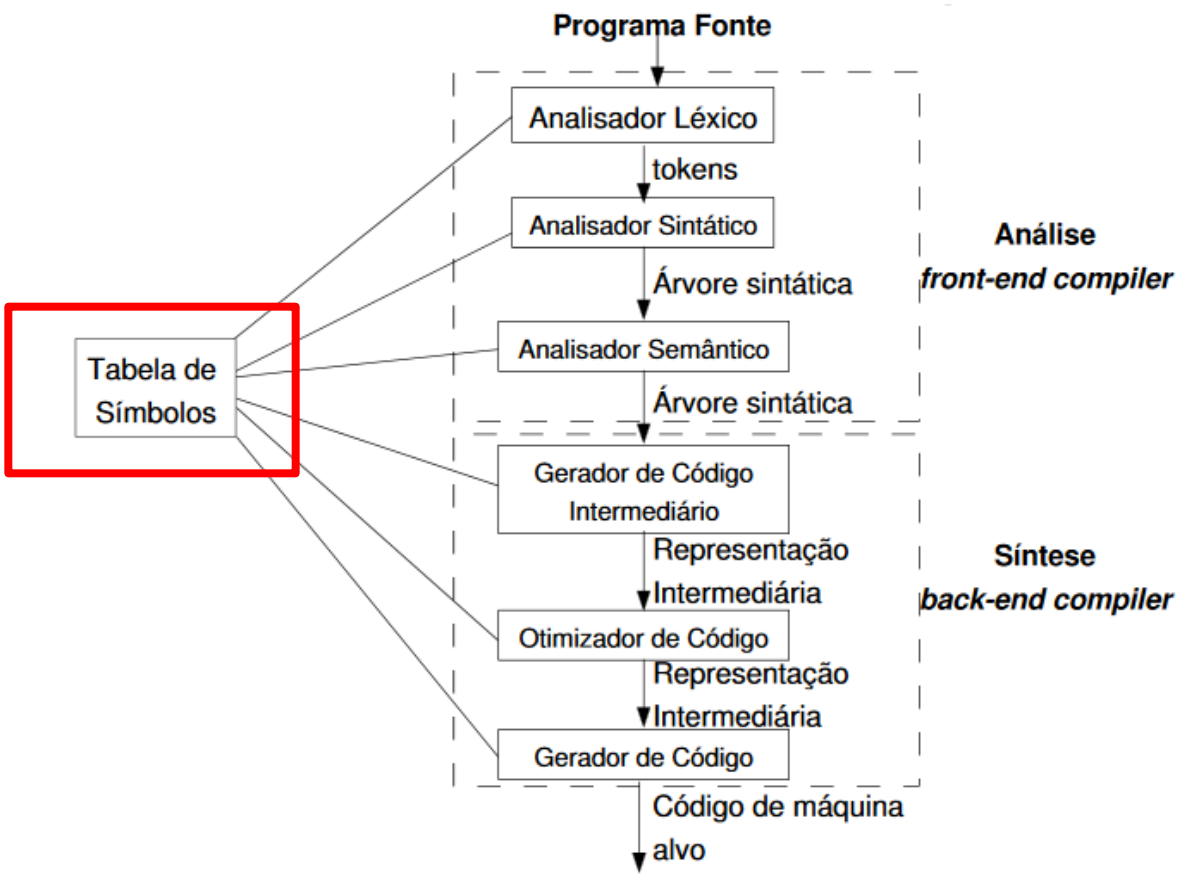
- O gerador de código:

```
t2 = id3 * 60.0
id1 = id2 + t2
```

Gerador de código ⇨

```
LDF  R2,  id3
MULF R2,  R2, #60.0
LDF  R1,  id2
ADDF R1,  R1, R2
STF  id1, R1
```

Fases de um Compilador



Fases de um Compilador

- Módulo de Gerencia da Tabela de símbolos:
  - Tem por função guardar informações a respeito de todos os nomes usados pelo programa e registrar informações importantes associadas a cada um, tais como seu tipo,tamanho, escopo, etc
  - A estrutura de dados usada para registrar essas informações é chamada tabela de símbolos
  - Por exemplo:
    - tipo do identificador
    - escopo: onde o identificador é válido no programa
    - se for um procedimento ou função: número e tipo dos argumentos, forma de passagem dos parâmetros e tipo do resultado.

Fases de um Compilador

- Tabela de símbolos:

	nome	tipo	
1	`position`	-	...
2	`initial`	-	
3	`rate`	-	
...			

Usada e atualizada em várias etapas da compilação.

Fases de um Compilador

- Manipulador de Erros:
  - É ativado sempre que for detectado um erro no programa fonte
  - Ele deve avisar o programador da ocorrência do erro emitindo uma mensagem, e ajustar-se novamente à informação sendo passada de fase a fase de modo a poder completar o processo de compilação

Fases de um Compilador

- Observações:
  - Os módulos de gerência de tabela de símbolos e manipulação de erros interagem sempre com todos os outros módulos do compilador
  - Na construção de um compilador, uma ou mais fases são combinadas em um único bloco chamado passo. Um passo lê o programa fonte ou a saída gerada pelo passo anterior, fazendo as transformações específicas de suas fases, gravando sua saída em um arquivo temporário que vai ser lido pelo passo posterior
  - O número de passos e o agrupamento de fases em cada passo são geralmente definidos em função da estrutura da LP e das características da arquitetura para a qual o compilador vai gerar código

Histórico

- Primórdios: linguagem de máquina por sequências de 0's e 1's
- Linguagens de montagem
- Linguagens de alto nível: Fortran, Cobol e Lisp
- DSLs (Linguagem Específica de Domínio).
  - Ex: arquivos de configuração Lua, SQL, etc

Impacto nos Compiladores

- O projeto de compiladores deve acompanhar os avanços nas linguagens de programação e nas arquiteturas de computadores
  - Novos algoritmos
  - Novas representações

- É possível gerar código objeto ótimo a partir de um programa fonte?
  - Problema indecidível -> uso de heurísticas

Aplicações

- Implementação de linguagens de programação de alto nível
  - Novas características para facilitar a vida do programador
  - Arranjos, estruturas, loops, métodos, herança, etc.
- Otimização para arquiteturas de computador
  - Paralelismo
  - Hierarquia de memória
- Projetos para definir novas arquiteturas de computadores

Aplicações

- Tradução de programas
  - Tradução binária
  - Simulação compilada
  - Linguagens de consultas de BDs

Ferramentas de Produtividade de Software

- Ferramentas de inspeção de código
  - FindBugs
    - Ferramenta popular voltada para a verificação de erros desenvolvida pela University of Maryland
  - PMD
    - Ferramenta voltada tanto para a verificação de regras de estilo quanto para a verificação de erros
- Descoberta de erros
  - Klocwork Developer
    - Além de encontrar defeitos, ela também procura por vulnerabilidades de segurança e ainda executa métricas e análises arquiteturais
- ‘